

Problem S1. Positioning Peter's Paintings

For two rectangles, only two layouts need to be considered for an optimal axis-aligned bounding rectangle: place them side by side, or stack one above the other.

If painting 1 has dimensions $A \times B$ and painting 2 has dimensions $X \times Y$, then the side-by-side bounding rectangle has length $A + X$ and height $\max(B, Y)$. The stacked bounding rectangle has length $\max(A, X)$ and height $B + Y$. Therefore the answer is

$$\min(2(A + X + \max(B, Y)), 2(\max(A, X) + B + Y)).$$

Complexity. The solution uses constant time and memory.

Problem S2. Cryptogram Cracking Club

The cipher describes a repeated pattern by run-length encoding. For small subtasks, the pattern can be explicitly expanded and the desired character can be found by indexing into the repeated string.

For larger subtasks, first compute the total pattern length $|S|$. Since the pattern repeats forever, replace the queried position c by its position modulo $|S|$. If the expanded pattern is still too large to store, keep the compressed list of pairs (character, count). Walk through the pairs, subtracting counts from the target position until the target lies inside one run; the corresponding character is the answer.

Use 64-bit integers for counts and positions.

Complexity. The compressed solution runs in $O(K)$ time, where K is the number of runs, and uses $O(K)$ memory.

Problem S3. Pretty Pens

For each colour c , let m_c be the prettiest pen of that colour. Without changing colours, the score is

$$T = \sum_c P_{m_c}.$$

If one recolouring helps, it is enough to move the best pen that is not currently one of the m_c 's into the colour whose current maximum is smallest. Thus the answer after each update is

$$\max(T, T - \min_c P_{m_c} + \max_{p \notin \{m_c\}} P_p).$$

Maintain three kinds of ordered sets: for each colour, all pens of that colour; the set of current colour maxima m_c ; and the set of all other pens. When a pen changes colour or prettiness, update the affected colour sets, adjust which pens are maxima, update T , and recompute the formula above.

Complexity. With balanced binary search trees or multisets, the total running time is $O((N + Q) \log N)$.

Problem S4. Floor is Lava

Model the cave as a graph whose edges have required chilling levels. A state can be represented by the last edge traversed. Moving from one edge e to an adjacent edge f costs $|w(e) - w(f)|$, because the boots must be adjusted from level $w(e)$ to level $w(f)$. Add a dummy starting edge at node 1 with weight 0, and run Dijkstra on this state graph until reaching any edge incident to node N .

If a vertex has high degree, connecting every pair of incident edges would create too many transitions. Sort the incident edges by weight and connect only neighbouring weights in this order. A transition that skips over intermediate weights is never better than walking through those intermediate states, so these adjacent transitions preserve all shortest paths while keeping only $O(M)$ transitions overall.

Complexity. The constructed state graph has $O(M)$ vertices and edges, so Dijkstra runs in $O(M \log M)$ time after sorting adjacency lists.

Problem S5. To-Do List

Sort jobs by release time $r_i = s_i - 1$. An exchange argument shows that some optimal schedule processes jobs in nondecreasing release time: if two adjacent jobs are out of order, swapping them cannot violate release times or increase the makespan.

For a fixed sorted list, the optimal makespan is

$$\max_i \left(r_i + \sum_{j \geq i} t_j \right).$$

This suggests a segment-tree summary for an interval: store (ans, sum) , where sum is the total processing time in the interval and ans is the best value of the expression within the interval. When concatenating left interval A and right interval B ,

$$sum(AB) = sum(A) + sum(B), \quad ans(AB) = \max(ans(A) + sum(B), ans(B)).$$

Build the segment tree over possible release times rather than over a changing list of jobs. Updating a job modifies one release-time leaf, and the root summary gives the current answer.

Complexity. With maximum release time s_{max} , initialization and updates take $O(s_{max})$ memory and $O(\log s_{max})$ time per query.